

72.1 → 89.6

The original queue of 90 items is done, the 1,730-line Home screen is split, and ten real bugs turned up along the way — most of them things a user would have noticed. A fresh review of the code that now ships raised 85 more findings, and all 85 are now fixed too. The twenty judgement grades have been re-measured against the code as it stands rather than carried forward. A few things still need checking on a real phone.

STRICT SCORE	MECHANICAL SCORE	FINDINGS FIXED	APP TESTS
72.1 → 89.6	79.7 → 96.5	175	282 → 644
re-graded, not carried forward	file sizes, duplication, lint	the original 90, then all 85 raised since	plus backend, 55 → 96

BUGS FOUND WHILE CLEANING

These weren't on the list. They surfaced because splitting a file forces you to read it properly. Each one is described by what you'd have seen, not by what the code did.

- USERS SAW IT

Every comment showed the wrong time

Comment timestamps were saved in UTC but read back as if they were local time, so every comment was off by your timezone offset — three hours out in East Africa, eight in California. Timestamps are now stored the same way as bookmark and download dates.
- USERS SAW IT

The AI comparison feature was switched off in every correct build

Bookmarks refused to generate a comparison unless an OpenAI key was present in the app. That key had already been moved to the server for security — so in any properly configured build the variable is empty and the feature simply declined to run. The check is gone; the server owns that decision.

USERS SAW IT

One phone could like the same solution over and over

On the Bookmarks screen, every tap of the thumbs-up added another like, with no record that you'd already liked it. Everywhere else in the app it toggles. All four screens now share one implementation, so the count can't be run up.

USERS SAW IT

Unknown cost and complexity were labelled "Moderate"

Three different places mapped an unrecognised value to the middle of the scale, so a guess was presented as a specific answer — including in the documents users export and keep. The drawer now shows a dash, and exports say "Not specified".

USERS SAW IT

Comparison summaries could stop mid-word

The AI was given a hard length limit but only *asked* to be brief, so a long reply got sliced wherever the budget ran out and the fragment was shown as written. It now trims back to the last finished sentence.

USERS SAW IT

Reduced motion was ignored while the app started

The setting is read from storage, and until it arrived the app assumed motion was fine — during the launch sequence, which is when it runs the most animation at once. Someone who had asked for reduced motion got the animations anyway. It now stays still until it knows otherwise.

COULD HAVE HUNG

A search with no results could spin forever

When a search finds nothing, the app fetches hotlines to offer instead. If the server ever reported "more results available" while returning an empty page, that fetch looped indefinitely, asking for the same page over and over. An empty page now ends the loop regardless of what the flag says.

QUIET DATA LOSS

Liking something while more results loaded undid the like

The "load more" handler held a stale copy of the results list, so appending the next page reverted any like registered in between.

QUIET DATA LOSS

A summary section could be swallowed by the one above it

The comparison summary is split into Use Case, Approach and Complexity/Cost. If one heading came directly after another with no blank line, the second was never recognised and its text was folded into the previous section. Two more spellings the AI commonly uses were also unrecognised and rendered as raw capitals.

SPEED

Opening a category made 152 database calls instead of 7

Loading a list of 30 solutions issued five separate database queries for every single row. It now fetches everything in seven queries no matter how many rows come back.

THE SECOND PASS

The fresh review raised 85 findings. Twelve pieces of work were picked first, on one rule: a user could be harmed by it, or it costs almost nothing. Eight of those were real bugs, described below by what you'd have seen. The remaining sixty were then done as well — that section follows.

USERS SAW IT

Filters silently returned the wrong solutions

A filter value containing % or _ was handed to the database as a wildcard instead of as text — so _ quietly matched *any* character in its place and returned solutions that don't belong in that slice. The fix already existed in the codebase; it had been wired into two of the ten places that needed it. All ten now use it. Nothing was ever injectable, and no current category name contains those characters, so this was a trap waiting on the next piece of data rather than a live fault.

USERS SAW IT

Dictation could stick on "transcribing" forever

Every other call to the server gives up after a set time. The one that uploads your recording had no limit at all — so a stalled connection left the microphone in the transcribing state permanently, with no error and no way to retry short of restarting the app. It now gives up after sixty seconds, which is generous because it's uploading a file, and tells you it timed out.

USERS SAW IT

Both heat maps showed a spinner that never stopped

If either heat map failed to load, the failure was written to the developer console and nothing else happened — and "no data yet" is exactly how both of them draw a load still in progress. So a failure looked identical to a slow load, forever. Both now say the load failed and offer a retry that genuinely retries.

USERS SAW IT

An empty search could leave the help section spinning

Search returns nothing → the app starts fetching hotlines → you search again before it finishes → that second search is also empty. The interrupted fetch never cleared its loading flag and never released the "only fetch once" latch, so the help section sat spinning and could never try again for the rest of that visit.

USERS SAW IT

The results list showed "Loading more..." when it wasn't

The footer spinner was wired to "there are more pages" rather than to "a page is loading right now", so it was pinned on for the entire time you scrolled a multi-page result. The correct value was already being calculated and simply never read.

WAITING TO BREAK

Two database modules imported each other

The only circular import in the app, and it existed for one three-line helper that had been left behind in the wrong file — the file that owns it never even used it. It works today by accident of load order. Moved to its own small module, with tests.

SAID ONE THING, DID ANOTHER

A counting function promised a safety limit it never applied

Its documentation said a broad filter would stop counting at a ceiling rather than scan the whole table. No ceiling was ever passed. Here the code was right and the description was wrong: the catalogue is a fixed ~3,000 rows, and truncating the total would make the list stop paging early and hide real results. So the description now says what actually happens and why, and the truncation signal it used to throw away is read and logged.

DEAD CODE**A billing guard that could never fire**

Search carried a documented protection against re-running an identical query and re-billing the AI call. Both callers overrode it, and the blur-and-refocus events it was written for never call search at all. It also fought the two things that *should* re-run: submitting the same text again is how you retry after a failure. Removed, rather than left looking like a live safeguard.

HOUSEKEEPING**Half the repository was a generated test report**

`coverage/` — 104 of 221 tracked files, 2.8 MB of generated HTML — was committed, wasn't ignored, and described a folder layout the refactor had already removed. Every coverage run dirtied the working tree. Now ignored and removed from tracking.

HOUSEKEEPING**A setup step nobody could have followed**

The backend can require a shared token on every request. The app sends one. It appeared in no example file and no instructions — so anyone who set the token on the server and followed the setup guide got a blanket "unauthorized" on search, dictation and both summary features, with nothing anywhere to explain it. Now documented on both sides, with a troubleshooting entry.

HOUSEKEEPING**The README described a layout that no longer exists**

It pointed four times at a file the refactor deleted, and had no entry at all for three of the folders the app is now organised around. The architecture document was already correct, so the README was brought in line with it.

TIDYING**One block of wiring, written out three times**

The twelve lines that connect a solution card to the like/bookmark/download behaviour were byte-for-byte identical in all three Home views, so adding a button to a card meant editing three files and hoping they stayed in step. Now one component.

THE THIRD PASS – THE OTHER SIXTY

The remaining sixty findings were deferred rather than rejected, and then done. Almost none was a bug. What they had in common is more interesting than any one of them.

In nearly every case the right version of the code already existed next to the wrong one. A helper that escapes special characters in filter values was wired into two of the ten places that needed it. A filter list that had been converted to a clean lookup table sat directly above an 88-line hand-written version doing the same fifteen categories. A grouping helper existed in one database file and was hand-written four times in the file next door. These were not gaps in someone's knowledge — they were jobs stopped halfway.

The second pattern: **comments that promised behaviour the code didn't deliver.** "Every write reports whether it succeeded" — four of six places ignored that report. "That path is bounded" — no limit was ever applied. "Nothing else calls the network directly" — something else did. Three written contracts, three of them false.

What changed the shape of it

one network client	four hand-rolled copies of the same request harness became one; there is now exactly one place in the app that talks to the server
one filter panel state	the challenge and type halves were written out twice; now one shared piece of logic serves both 492 → 388 lines
components stop reaching into the database	four components did their own database and network work; that now lives with the rest of the data layer, so a component is a component
storage promises kept	a failed save no longer shows as saved: likes, downloads and accessibility settings all report and surface their own failures

One fix that introduced a bug — and how it was caught

Making page-loading safe against overlapping searches introduced a new fault: a page that got overtaken left the list permanently believing it was still loading, which silently disabled scrolling for more results for the rest of the session.

It was found by the independent review that produced the scores below, reproduced with a test, and fixed. Worth saying plainly, because it is the honest cost of a change this size — and the argument for having something other than the author look at it.

THE BIG SPLIT

Four files were doing several jobs each. Nothing about how the app behaves changed — the point was that these parts could not be tested or understood separately.

HomeScreen.js 1,730 lines, 45 pieces of state → 263

```
HomeScreen.js      the shell: which half you're on, and what spans both
  home/SearchMode.js  the landing page and the results list
  home/ExploreMode.js stats, category grids, hubs, recent solutions
  home/DrilldownView.js a filtered slice of the catalogue
+ five hooks holding search, explore data, drilldown, help lookup
  and the shared bookmark/like/download behaviour
```

db.js 1,040 lines → 355, as five modules

```
connection.js  opening the database, and the tables this app adds
db.js          the read-only catalogue queries
enrich.js      turning a raw row into a solution
engagement.js  likes, comments, cached summaries – the only writes
heatmaps.js    the two grids
```

BookmarksScreen.js 585 → 215 · **FilterPanel.js** 709 → 492

The 350-line comparison feature moved out of Bookmarks, where it had nothing to do with listing bookmarks. In the filter panel, three separate patterns had each been written out two to four times — the category pickers, the readiness and adoption sliders, and the toggle-chip grids. Several of those copies had already drifted apart from each other.

THE TESTS THAT WERE IMPOSSIBLE BEFORE

The previous session wrote six tests that drive a search from typing to results, watched them fail, and deleted them — the submitting button only appeared after an interaction locked inside the screen's own state, so no test could reach it. That was the argument for splitting the file.

Those tests are back and passing. A search now runs end to end in the suite: typed query, backend call, ordered results, error, retry, and the empty-result hotline state. The suite went from 282 to 499 in that pass and stands at 514 after the second one, and the new tests are mostly of things that were previously only assumed — including one that pins the filter contract across the three separate modules that each interpret it, which nothing had checked before.

WHAT THE SCORE ACTUALLY SAYS

Both halves have now been measured against the code as it stands. The mechanical quarter is counted by the tool. The other twenty dimensions are judgement calls, so they were re-graded from scratch by twenty independent reviewers — one per dimension, each told to assess the code on its current merits and explicitly not to anchor on the previous number or credit a recent refactor.

SCORE	AT THE START	NOW	BASIS
Strict	72.1	89.6	re-graded · target was 85
Overall	75.3	90.3	re-graded
Mechanical	79.7	96.5	measured

Every dimension now sits between **84.0 and 92.0**, against a spread of 64 to 82 at the start. The two that were dragging hardest are the two that moved most: test strategy 64.0 → **84.0** and authorization consistency 69.5 → **90.5**. The highest are initialization coupling at 92.0 and incomplete migration at 91.5 — both of which had been near the bottom.

The lowest two are now test strategy and type safety, both at 84.0. That is a much narrower band than before, which is itself the useful signal: there is no longer one weak area dragging the rest.

Why this number is worth more than the last one

The previous report carried 87.5 with a caveat: three quarters of it was judgement grades from *before* the work, measuring a codebase that no longer existed. That caveat is gone. This 89.6 was produced by twenty fresh reviews of the code that ships today.

It also went up by less than the amount of work would suggest — the reviewers found 87 new things while they were looking. That is the system working as intended rather than a disappointment: a review that returns nothing is a review that wasn't looking.

WHAT THE NEW REVIEW FOUND

The twenty reviewers raised **87 new findings** while grading. None is a correctness bug except the one described above, which is already fixed.

This is the third time this has happened, and it is worth being clear about why rather than treating it as bad news. A real review of sixty-odd files never comes back empty. The first review found 90 things, the second 85, this one 87 — but they are not the same 87. The earlier rounds found duplicated code, contracts that lied, and a data layer no test had ever executed. This round found nothing of that severity: the largest clusters are unfinished renames, comments describing where code used to live, and sibling files that solve the same problem two different ways.

Where they concentrate is itself informative. **19 of the 87 touch the main database file and 16 touch the Home screen** — the two files that have absorbed the most change. The most repeated single observation, made independently by six reviewers, is that the Home screen still loads its two heat maps directly instead of through the data layer every other screen uses. That is the one structural item left.

```
desloppify show review --status open
```

41 are rated high confidence, 41 medium, 5 low. They are recorded, not swept, and none of them blocks testing what is on the branch now.

WHERE THE TESTS STAND NOW

Test strategy was the lowest-scoring dimension at 64.0 and the one piece of work that was a genuine gap rather than an unfinished job. It is now 84.0.

Three things had no real tests. The **database layer** — every query behind every list — was replaced by a stand-in everywhere it appeared, so no test had ever run a real query. The **network client** was mocked wholesale. The **server's AI features** were tested only with the key switched off, which exercises the fallback and never the feature.

All three are now covered, and the database tests run against a real database rather than a stand-in — including a deliberately planted decoy row that a filtering bug would wrongly match, so the test fails if that protection is ever removed.

Three things had also been making the numbers untrustworthy. Coverage was reported as 67.5% when the real figure was 52.9%, because untested files were being left out of the report rather than counted as zero. `npm test` didn't run the server's tests at all. And the test setup was discarding every warning, so problems could scroll past unseen. All three fixed; honest coverage is now **68.5%** of lines.

The remaining gap is by feature rather than by layer: the Downloads and Settings screens, the file export, and dictation are the thinnest. That is the next honest piece of work.

Everything below passes in the test suite, but the suite cannot prove these particular things. They're worth ten minutes on a phone before shipping.

What *has* been proved automatically, so you know where the line is: the app compiles into a shippable bundle for both iOS and Android, which means every import across the restructuring resolves. The whole app — the real provider tree, the navigator and all four tabs — now mounts in the test suite and survives being navigated end to end; that check was added specifically because nothing had ever rendered the app's root, and it was confirmed to fail when a screen is deliberately broken.

What that still doesn't prove is anything involving the actual device: the file system, the real database, the microphone, and how it all looks. Hence the list.

```
git fetch && git checkout krishnav8
npm install          # two packages were removed, so this matters
npm start
# the AI search, comparison and dictation features also need:
cd backend && npm install && npm run start
```

First launch on a clean install — the one that matters

The app copies its bundled database into storage the first time it runs. That code moved to the current Expo file API, and **no test can verify it**: every test replaces the file system with a stand-in, so passing tests only prove the new code agrees with a stand-in I also wrote. If it's wrong, the app can't open its database at all — it fails at startup, not subtly.

Delete the app, install fresh, launch. If the catalogue loads, it works. The change is isolated in its own commit, so it's a clean revert if not.

02 Export a solution to a file

The other half of the same file-system change. Tap download on any solution and confirm the share sheet opens with a readable file.

03 Navigation in a release build

The app was disabling native navigation screens in every build to work around a problem that only affects Expo Go. That's now limited to Expo Go, so a release build gets native screens for the first time. Worth tapping through the tabs and back gestures once.

04 Text size set to Large

About 150 pieces of text that previously ignored this setting now follow it.

That's the intended fix, but it means some screens will lay out differently at the large size than they ever have.

DELIBERATELY LEFT ALONE

- **The OpenAI key is out of the app, but not yet rotated.** Nothing in the app reads it any more, and the unused `EXPO_PUBLIC_OPENAI_API_KEY` entry has been deleted — the same key already lives in `backend/.env`, where it belongs, so nothing was lost. What remains is that any build made while it was in the app still contains it, on whatever devices have those builds. Only rotating the key at OpenAI fixes that, and your standing instruction was not to, so I haven't.
- **The backend is not locked down yet.** It now supports requiring a client token, but that stays off until you set `API_CLIENT_TOKEN` on the server — turning it on without that would break builds already in users' hands. What changed this round is only that the setting is now *documented* on both sides, so turning it on is no longer a trap.
- **The 87 findings from the newest review are open.** Not marked won't-fix — recorded, so nothing has to be rediscovered. None is a correctness bug.
- `assets/db/atiokb.db` **was never touched**, and I checked after every commit.
- **On branch `krishnav8`** — 41 commits. Each is self-contained with the reasoning in the message, so anything here can be reverted on its own. Nothing was merged to `main`.

strict 89.6 · mechanical 96.5 · 644 app tests · 96 backend branch krishnav8 · pushed
database asset unchanged